Intro to Data Science - Lab 9

Copyright 2025, Jeffrey Stanton and Jeffrey Saltz Please do not post online.

Week 9 - Supervised Data Mining

```
# Enter your name here: Mars Schrag
```

Please include nice comments.

Instructions:

Run the necessary code on your own instance of R-Studio.

Attribution statement: (choose only one and delete the rest)

```
# 1. I did this lab assignment by myself, with help from the book and the professor.
```

Supervised data mining/machine learning is the most prevalent form of data mining as it allows for the prediction of new cases in the future. For example, when credit card companies are trying to detect fraud, they will create a supervised model by training it on fraud data that they already have. Then they will deploy the model into the field: As new input data arrives the model predicts whether it seems fraudulent and flags those transactions where that probability is high.

In these exercises we will work with a built-in data set called **GermanCredit**. This data set is in the ** caret ** package so we will need that and the ** kernlab ** package to be installed and libraried before running the following:

```
data("GermanCredit")
subCredit <- GermanCredit[,1:10]
str(subCredit)
```

```
#install.packages("caret")
#install.packages("kernlab")

library(kernlab)
library(caret)
```

```
## Loading required package: ggplot2

##
## Attaching package: 'ggplot2'

## The following object is masked from 'package:kernlab':
##
##     alpha

## Loading required package: lattice
```

```r
data("GermanCredit")
subCredit <- GermanCredit[,1:10]
str(subCredit)
```

```
## 'data.frame':    1000 obs. of  10 variables:
##  $ Duration                 : int  6 48 12 42 24 36 24 36 12 30 ...
##  $ Amount                   : int  1169 5951 2096 7882 4870 9055 2835 6948 3059 5234 ...
##  $ InstallmentRatePercentage: int  4 2 2 2 3 2 3 2 2 4 ...
##  $ ResidenceDuration        : int  4 2 3 4 4 4 4 2 4 2 ...
##  $ Age                      : int  67 22 49 45 53 35 53 35 61 28 ...
##  $ NumberExistingCredits    : int  2 1 1 1 2 1 1 1 1 2 ...
##  $ NumberPeopleMaintenance  : int  1 1 2 2 2 2 1 1 1 1 ...
##  $ Telephone                : num  0 1 1 1 1 0 1 0 1 1 ...
##  $ ForeignWorker            : num  1 1 1 1 1 1 1 1 1 1 ...
##  $ Class                    : Factor w/ 2 levels "Bad","Good": 2 1 2 2 1 2 2 2 2 1 ...
```

1. Examine the data structure that **str()** reveals. Also use the **help()** command to learn more about the **GermanCredit** data set. Summarize what you see in a comment.

```r
str(subCredit)
```

```
## 'data.frame':    1000 obs. of  10 variables:
##  $ Duration                 : int  6 48 12 42 24 36 24 36 12 30 ...
##  $ Amount                   : int  1169 5951 2096 7882 4870 9055 2835 6948 3059 5234 ...
##  $ InstallmentRatePercentage: int  4 2 2 2 3 2 3 2 2 4 ...
##  $ ResidenceDuration        : int  4 2 3 4 4 4 4 2 4 2 ...
##  $ Age                      : int  67 22 49 45 53 35 53 35 61 28 ...
##  $ NumberExistingCredits    : int  2 1 1 1 2 1 1 1 1 2 ...
##  $ NumberPeopleMaintenance  : int  1 1 2 2 2 2 1 1 1 1 ...
##  $ Telephone                : num  0 1 1 1 1 0 1 0 1 1 ...
##  $ ForeignWorker            : num  1 1 1 1 1 1 1 1 1 1 ...
##  $ Class                    : Factor w/ 2 levels "Bad","Good": 2 1 2 2 1 2 2 2 2 1 ...
```

```r
help(GermanCredit)
```

```
## starting httpd help server ... done
```

```r
#9 numeric variables, 1 categorical variable, class
```

2. Use the **createDataPartition()** function to generate a list of cases to include in the training data. This function is conveniently provided by caret and allows one to directly control the number of training cases. It also ensures that the training cases are balanced with respect to the outcome variable. Try this: `trainList <- createDataPartition(y=subCredit$Class,p=.40,list=FALSE)`

```r
trainList <- createDataPartition(y=subCredit$Class,p=.40,list=FALSE)

#y: a vector of columns
#percentage of data that goes to training

#trainlist: indices of 400 data points from subCredit, keeping original class ratio
```

3. Examine the contents of **trainList** to make sure that it is a list of case numbers. With **p=0.40**, it should have 400 case numbers in it.

```
#print(trainList)
```

4. What is **trainList**? What do the elements in **trainList** represent? Which attribute is balanced in the **trainList** dataset?

5. Use **trainList** and the square brackets notation to create a training data set called ** trainSet ** from the **subCredit** data frame. Look at the structure of trainSet to make sure it has all of the same variables as **subCredit**. The **trainSet** structure should be a data frame with **400 rows and 10 columns**.

```
trainSet <- subCredit[trainList,]

table(trainSet$Class)
```

```
##
##  Bad Good
##  120  280
```

```
table(subCredit$Class)
```

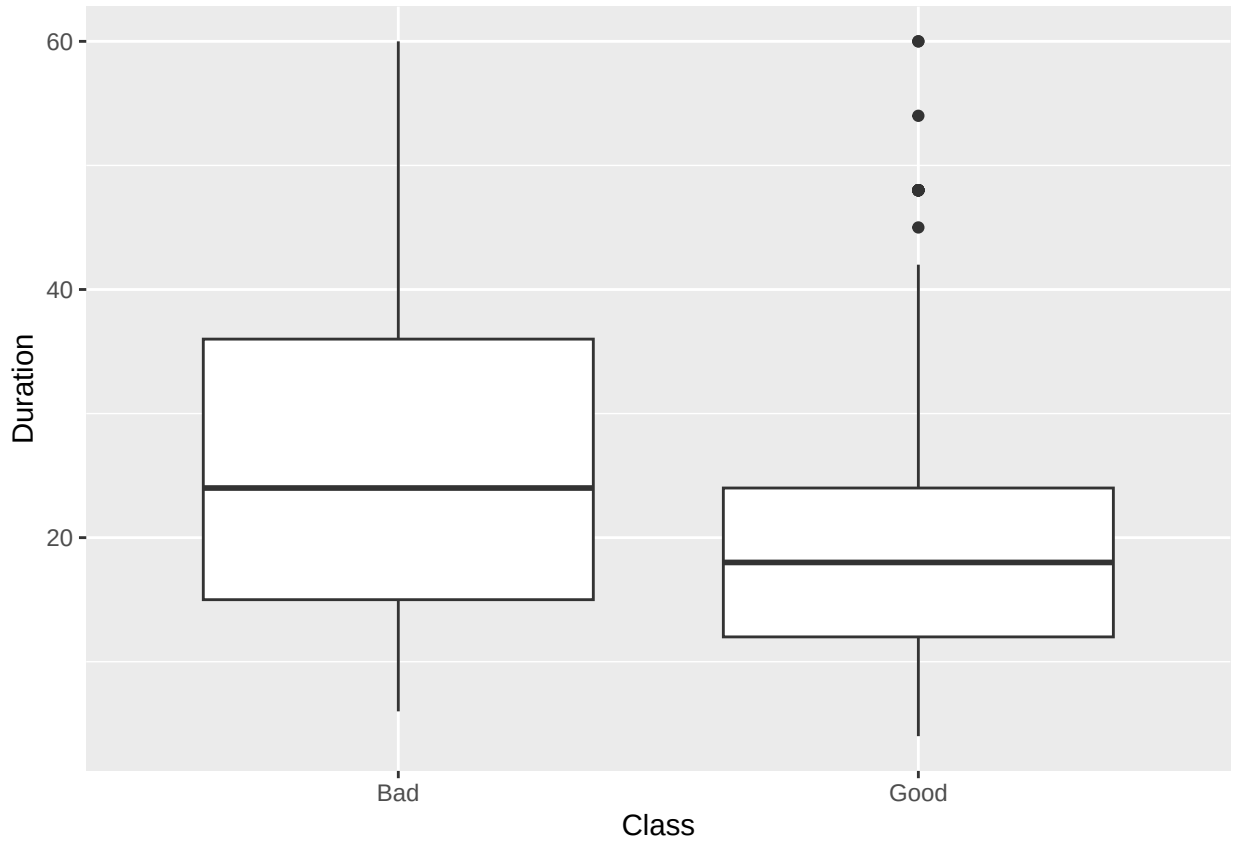
```
##
##  Bad Good
##  300  700
```

6. Use **trainList** and the square brackets notation to create a testing data set called ** testSet ** from the subCredit data frame. The **testSet** structure should be a data frame with **600 rows and 10 columns** and should be a completely different set of cases than **trainSet**.

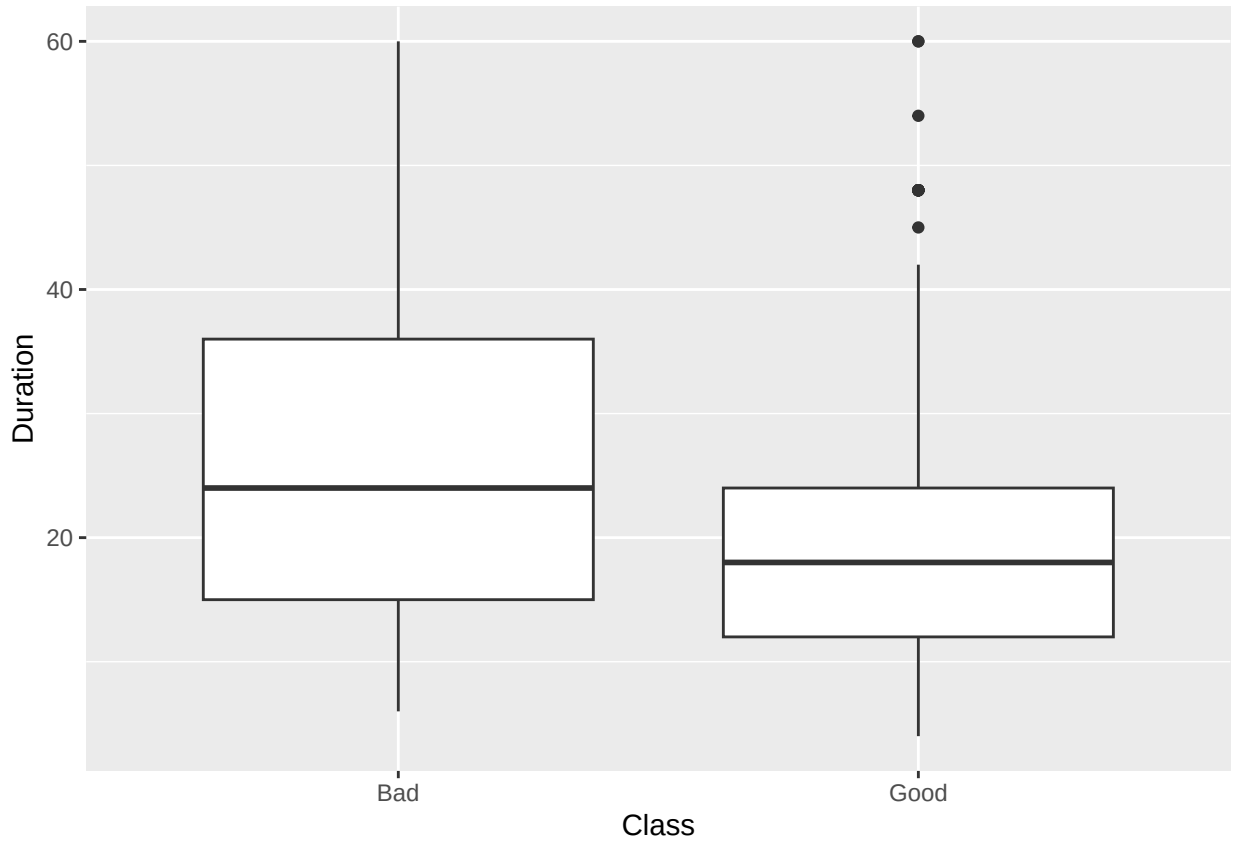
```
testSet <- subCredit[-trainList,]
```

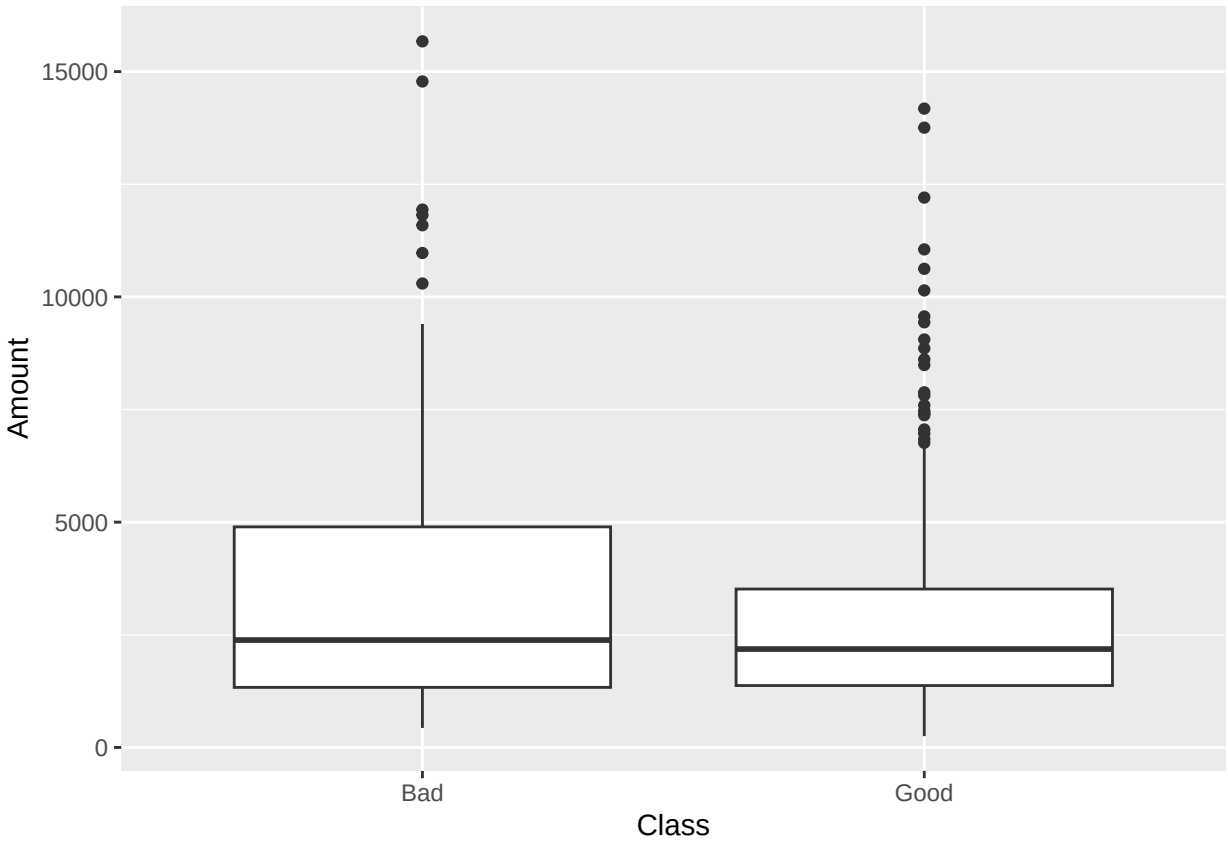
7. Create and interpret boxplots of all the predictor variables in relation to the outcome variable (**Class**).

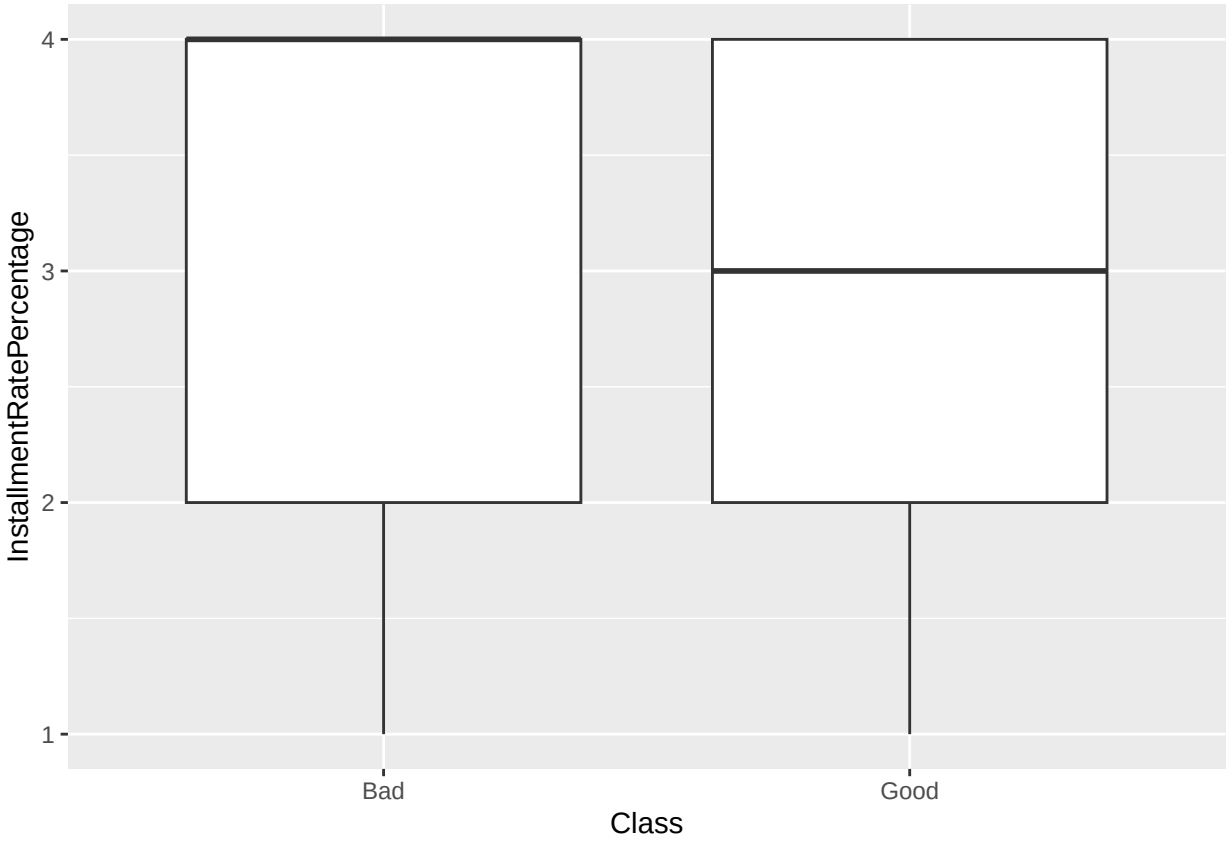
```
library(ggplot2)
ggplot(data=trainSet, aes(x=Class, y=Duration))+
  geom_boxplot()
```

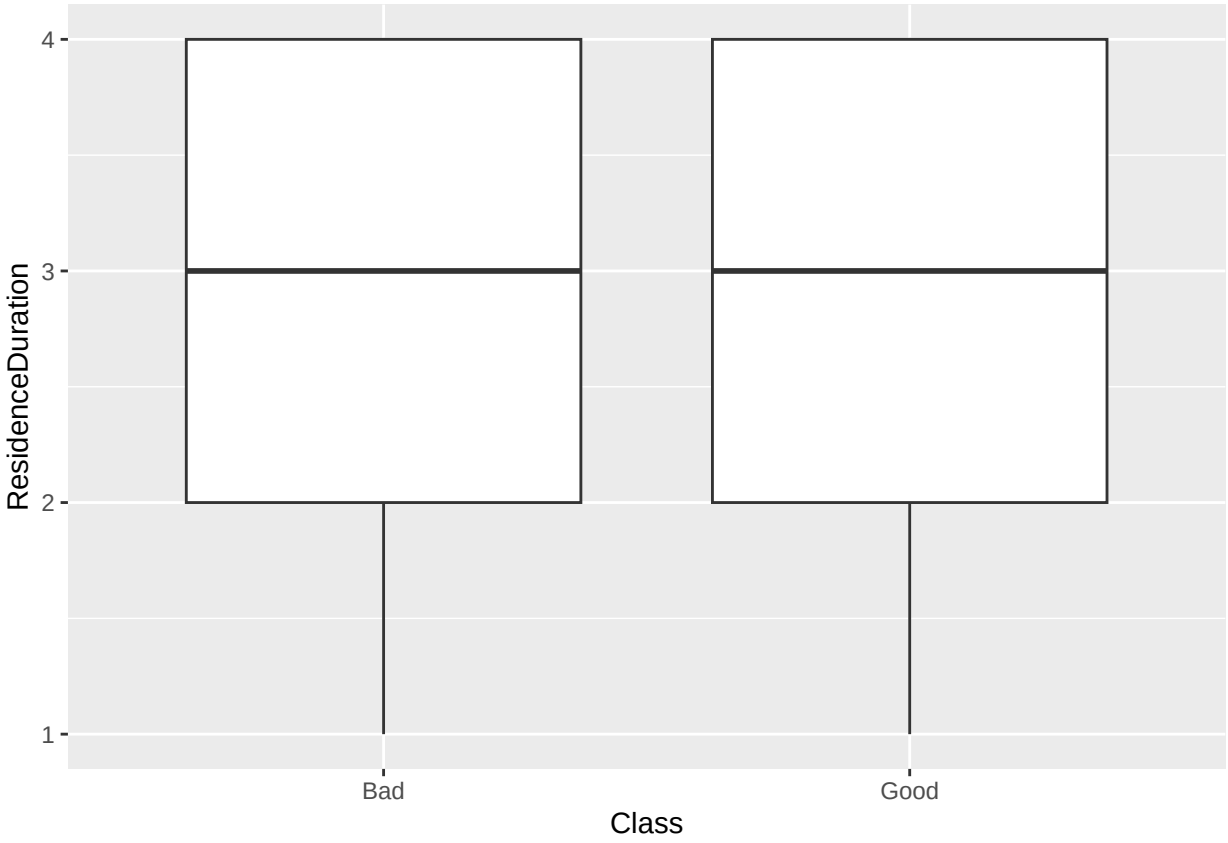


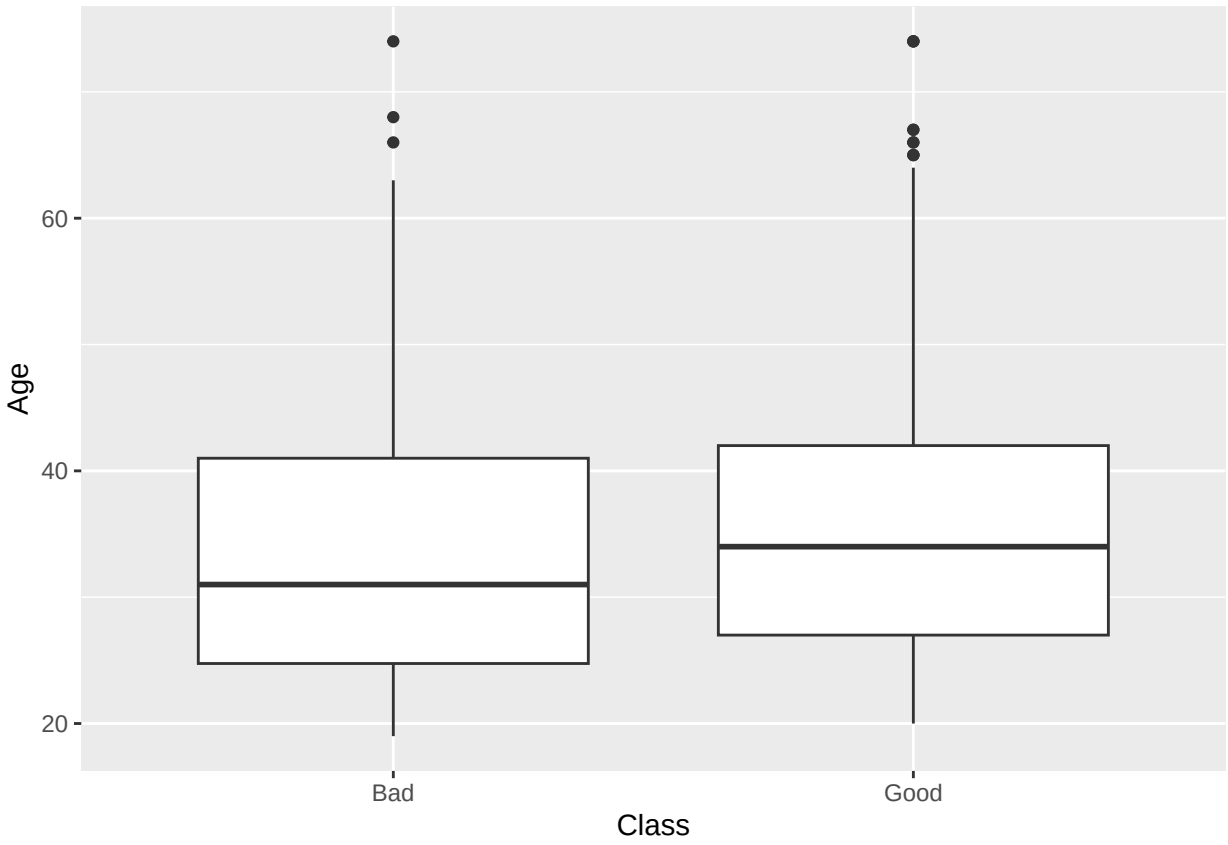

```
cols <- colnames(trainSet)
cols <- cols[cols!="Class"]

for (c in cols) {
  print(ggplot(data=trainSet, aes(x=Class, y=.data[[c]]))+
  geom_boxplot())
}
```

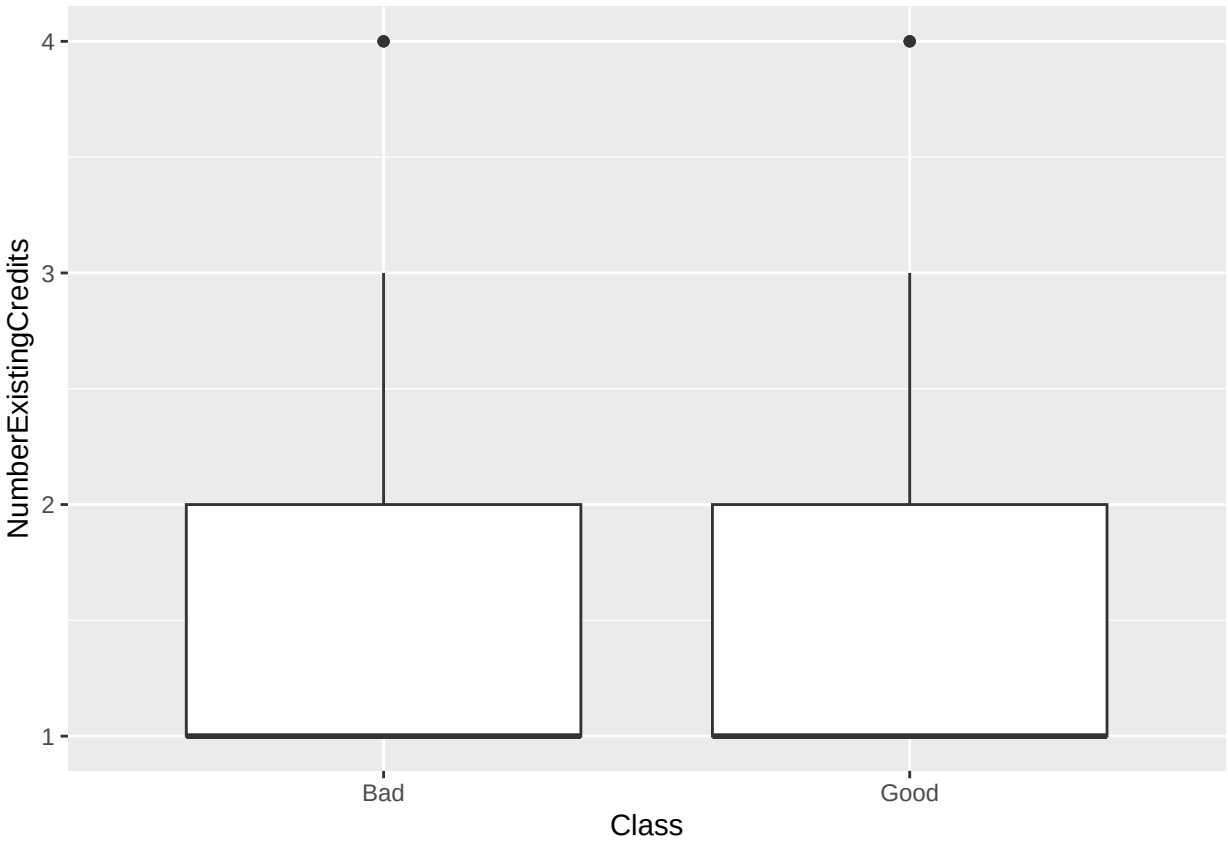

60
40
20
Duration
Bad
Good
Class

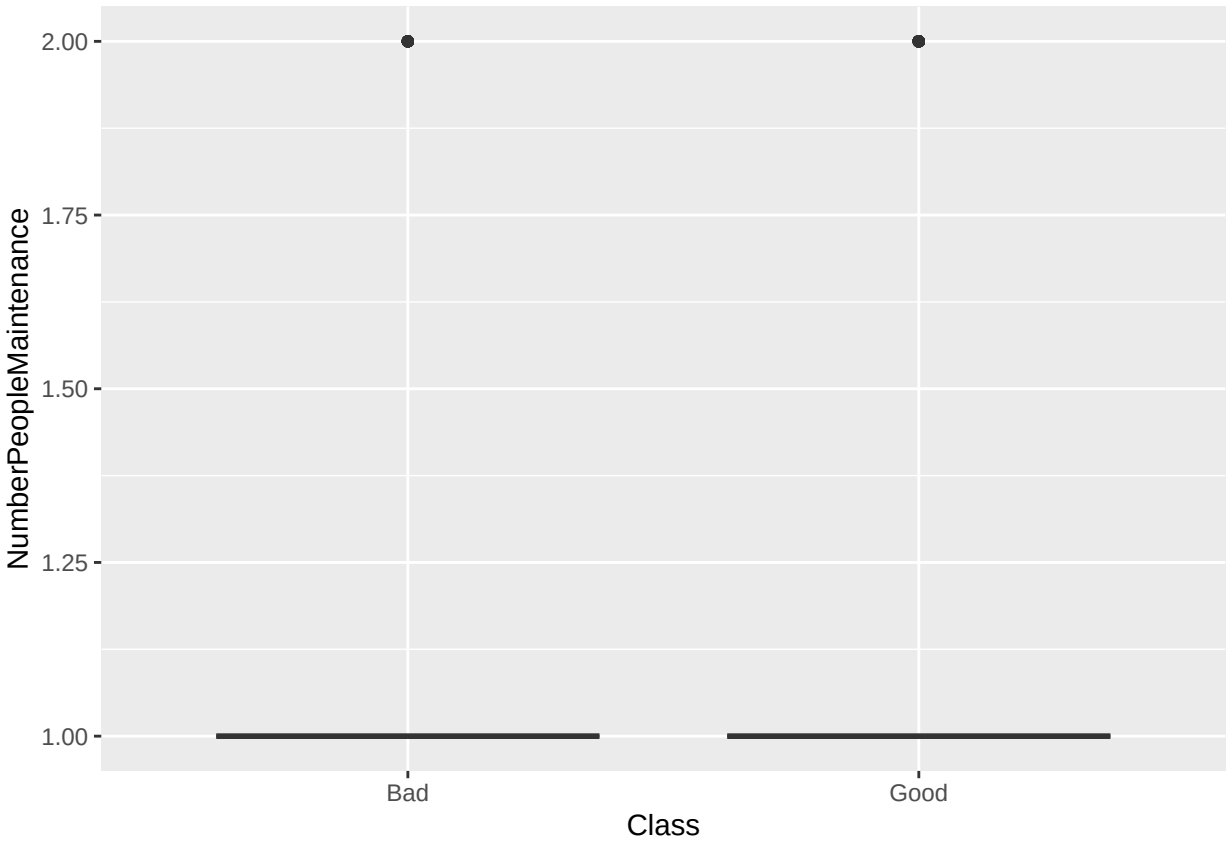

15000
10000
5000
0
Amount
Bad
Good
Class

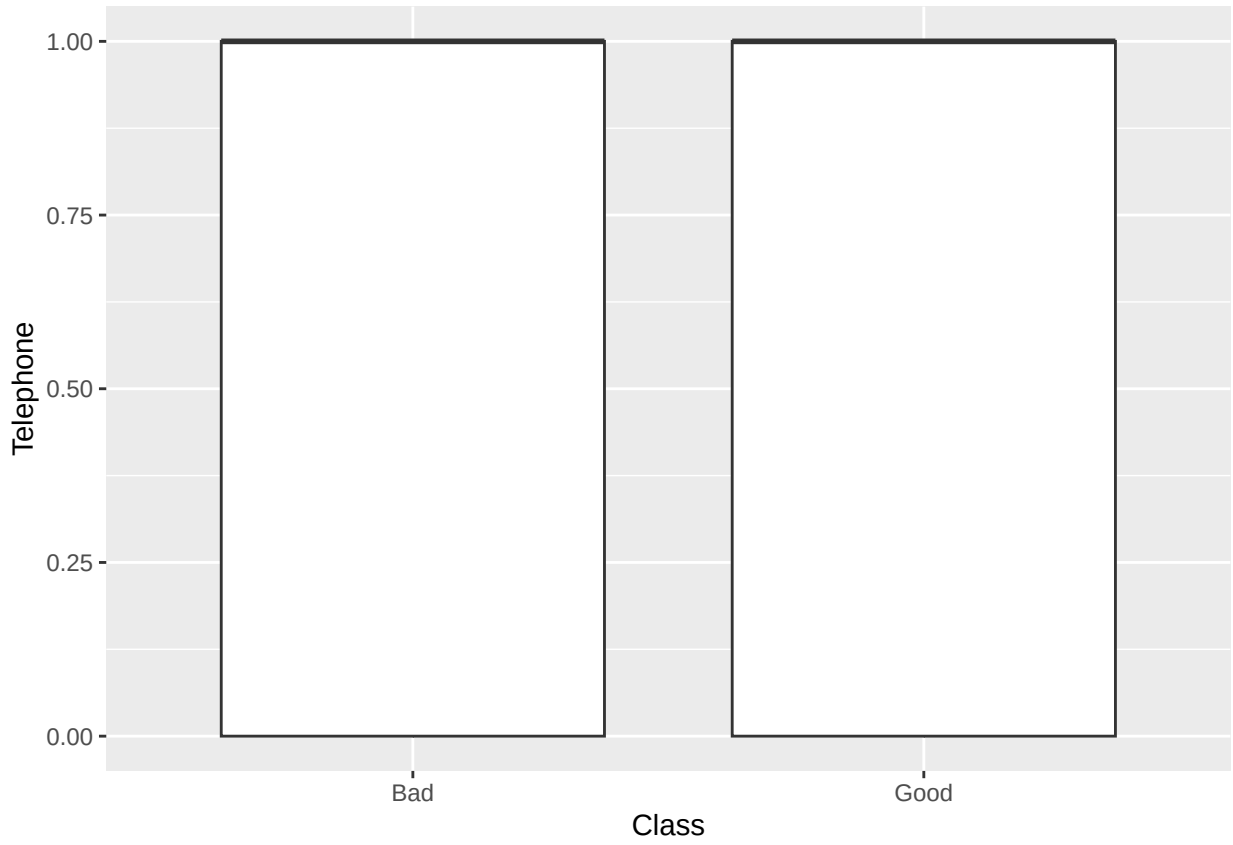

4
3
2
1
InstallmentRatePercentage
Bad
Good
Class

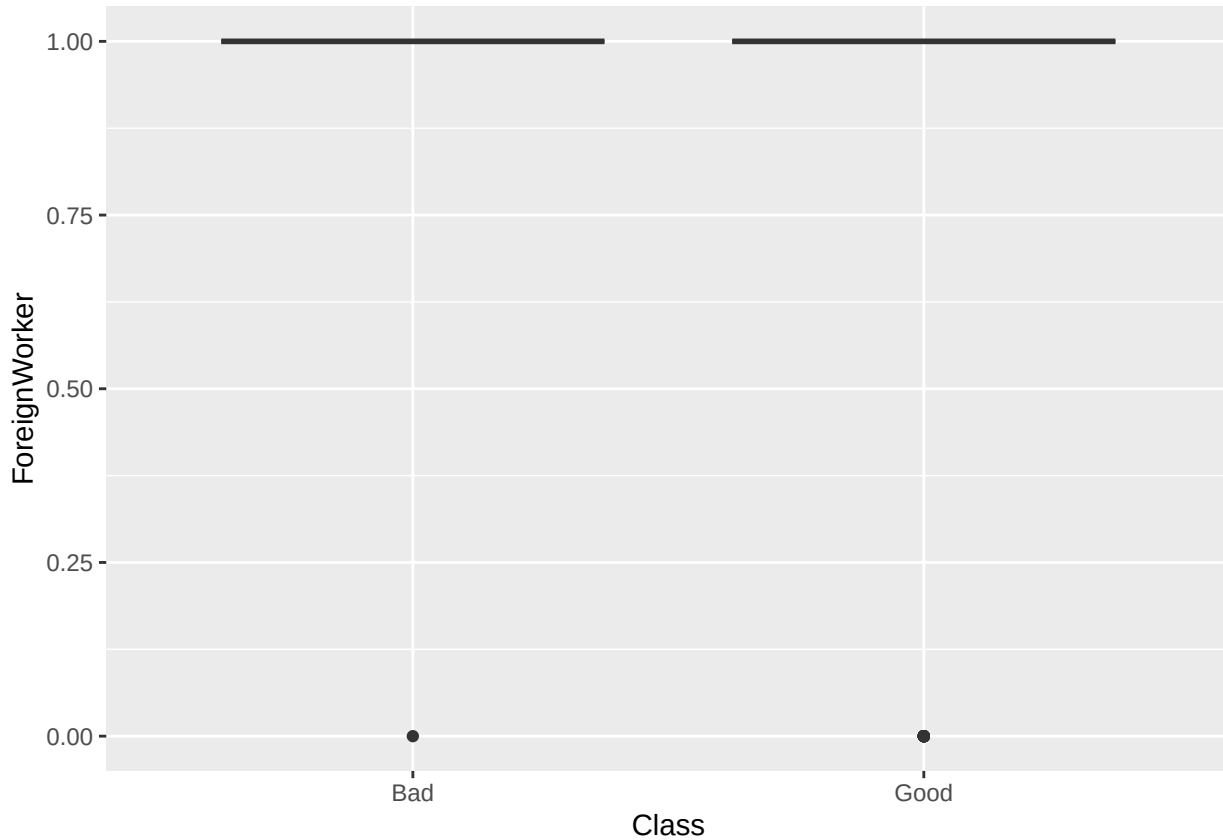

4
3
2
1
ResidenceDuration
Bad
Good
Class


60
40
20
Age
Bad
Good
Class


4
3
2
1
NumberExistingCredits
Bad
Good
Class


2.00
1.75
1.50
1.25
1.00
NumberPeopleMaintenance
Bad
Good
Class


1.00
0.75
0.50
0.25
0.00
Telephone
Bad
Good
Class



8. Train a support vector machine with the **ksvm()** function from the **kernlab** package. Make sure that you have installed and libraried the **kernlab** package. Have the **cost** be 5, and have **ksvm** do 3 **cross validations** (Hint: `try prob.model = TRUE`)

```
svm_model <- ksvm(Class~., data=trainSet, C=5, cross=3, prob.model  = TRUE)
```

9. Examine the ksvm output object. In particular, look at the **cross-validation error** for an initial indication of model quality. Add a comment that gives your opinion on whether this is a good model.

```
print(svm_model)
```

```
## Support Vector Machine object of class "ksvm"
##
## SV type: C-svc  (classification)
##  parameter : cost C = 5
##
## Gaussian Radial Basis kernel function.
##  Hyperparameter : sigma =  0.0905111192378017
##
## Number of Support Vectors : 272
##
## Objective Function Value : -955.2281
## Training error : 0.205
## Cross validation error : 0.299929
## Probability model included.
```

```r
#overfittingL theres a notable difference between the training error and the cross-validation error
```

10. Predict the training cases using the **predict()** command

```r
predict(svm_model, testSet)
```

```
##   [1] Good Good Bad  Good Good Good Good Good Good Good Bad  Good Good Good Good
##  [16] Good Good Good Good Good Good Good Good Good Good Good Good Bad  Good Good
##  [31] Good Good Good Good Good Good Good Good Bad  Good Good Good Good Good Good
##  [46] Good Good Good Good Good Good Good Good Good Good Good Good Good Good Good
##  [61] Bad  Good Good Good Good Good Good Good Good Good Good Good Good Good Good
##  [76] Good Good Good Good Good Good Good Good Good Good Good Good Good Good Good
##  [91] Good Good Good Good Good Good Good Good Good Good Good Good Good Good Good
## [106] Good Good Good Good Good Good Good Good Good Good Bad  Good Good Good Good
## [121] Good Good Good Bad  Good Good Good Good Good Good Good Good Good Good Good
## [136] Good Bad  Good Good Good Good Good Good Bad  Bad  Good Good Good Good Bad
## [151] Good Good Good Good Good Good Good Good Good Good Good Good Good Bad  Good
## [166] Bad  Good Good Good Good Good Bad  Good Good Good Good Good Good Good Good
## [181] Good Good Good Bad  Bad  Good Good Good Good Good Bad  Good Good Good Bad
## [196] Good Good Good Good Good Good Good Good Good Good Good Good Good Good Good
## [211] Good Good Good Good Good Good Good Good Good Good Good Good Good Good Good
## [226] Good Good Good Good Good Good Good Good Bad  Good Good Good Good Good Good
## [241] Good Good Good Good Good Good Good Good Bad  Good Good Good Good Good Good
## [256] Bad  Good Good Good Good Good Good Good Bad  Good Good Good Good Good Bad
## [271] Good Good Good Bad  Good Good Good Good Good Bad  Good Good Good Good Good
## [286] Good Good Good Good Good Bad  Bad  Good Good Good Good Good Good Good Good
## [301] Good Good Good Good Good Good Good Good Good Good Good Good Good Good Good
## [316] Good Good Good Good Good Good Good Good Bad  Good Good Good Good Bad  Good
## [331] Good Good Good Good Bad  Good Good Good Good Good Good Bad  Good Good Good
## [346] Good Good Good Good Good Good Good Good Bad  Good Good Good Good Good Good
## [361] Good Good Good Good Good Good Good Good Good Good Good Good Good Good Good
## [376] Good Good Good Good Good Good Good Good Good Good Good Bad  Good Good Good
## [391] Good Good Good Good Good Good Good Good Good Good Good Good Good Good Bad
## [406] Good Good Good Good Good Good Bad  Bad  Good Good Good Good Good Good Good
## [421] Good Good Good Good Good Good Good Bad  Good Good Good Good Good Good Bad
## [436] Good Good Good Good Good Good Good Good Good Good Good Bad  Good Bad  Good
## [451] Good Good Good Good Good Good Good Good Good Good Good Good Good Good Bad
## [466] Good Good Good Good Good Bad  Good Good Good Good Good Good Good Bad  Good
## [481] Bad  Good Good Good Good Good Good Good Bad  Good Good Good Good Good Bad
## [496] Good Good Good Good Good Good Good Good Bad  Good Good Bad  Good Good Good
## [511] Bad  Good Good Good Good Good Good Good Good Good Good Good Bad  Bad  Good
## [526] Good Good Good Good Good Good Good Bad  Good Good Good Good Good Good Good
## [541] Good Good Bad  Good Good Bad  Good Good Good Bad  Good Good Good Good Good
## [556] Good Good Good Good Good Bad  Good Good Good Good Good Good Bad  Good Good
## [571] Good Good Good Good Good Good Good Good Good Good Good Good Good Good Good
## [586] Good Good Good Good Good Good Bad  Good Good Good Good Good Good Good Good
## Levels: Bad Good
```

```r
testSet$Class
```

```
##   [1] Good Bad  Bad  Good Good Good Bad  Bad  Good Bad  Good Bad  Good Good Good
##  [16] Good Good Good Good Good Good Bad  Good Good Good Good Good Good Good Good
```

```
##  [31] Good Good Good Good Bad  Good Bad  Good Bad  Good Good Good Good Good Good
##  [46] Good Bad  Good Good Bad  Good Good Good Good Good Bad  Good Bad  Good Good
##  [61] Bad  Good Bad  Good Good Good Good Good Good Good Good Good Good Bad  Good
##  [76] Good Bad  Good Good Bad  Bad  Bad  Good Good Good Bad  Good Good Good Bad
##  [91] Good Good Good Good Good Good Good Good Bad  Good Good Good Good Good Good
## [106] Bad  Good Bad  Bad  Bad  Good Good Good Bad  Bad  Bad  Good Good Bad  Bad
## [121] Bad  Good Bad  Good Good Good Good Good Good Bad  Good Good Good Good Good
## [136] Good Bad  Bad  Bad  Good Bad  Good Good Bad  Bad  Bad  Good Good Bad  Bad
## [151] Good Good Good Good Good Good Good Good Good Bad  Good Good Good Good Good
## [166] Bad  Good Bad  Good Good Good Bad  Good Good Good Bad  Good Good Good Good
## [181] Bad  Good Good Good Good Bad  Good Good Good Bad  Bad  Good Bad  Good Good
## [196] Good Good Good Bad  Good Good Good Good Bad  Good Good Good Good Good Good
## [211] Bad  Bad  Bad  Good Bad  Good Good Good Good Good Bad  Bad  Good Bad  Good
## [226] Bad  Bad  Good Good Good Good Good Good Bad  Good Bad  Good Good Bad  Bad
## [241] Good Good Good Good Good Good Good Good Good Good Good Good Good Good Good
## [256] Good Good Good Bad  Good Bad  Bad  Good Bad  Good Good Good Good Good Bad
## [271] Good Bad  Good Good Bad  Good Good Bad  Good Bad  Good Good Good Good Bad
## [286] Good Good Good Good Good Good Good Good Good Bad  Bad  Good Good Good Good
## [301] Good Bad  Good Good Good Good Good Good Bad  Bad  Good Good Bad  Bad  Bad
## [316] Good Bad  Good Good Good Good Good Bad  Bad  Good Good Good Bad  Good Bad
## [331] Good Bad  Good Good Bad  Good Bad  Good Bad  Good Good Good Bad  Good Good
## [346] Bad  Good Bad  Bad  Bad  Good Bad  Good Bad  Good Good Bad  Good Bad  Good
## [361] Good Good Good Good Good Bad  Good Good Bad  Good Good Good Good Good Bad
## [376] Bad  Bad  Bad  Bad  Bad  Bad  Bad  Good Good Bad  Good Good Good Bad  Good
## [391] Good Bad  Good Bad  Good Bad  Good Bad  Good Bad  Good Bad  Good Good Good
## [406] Good Bad  Bad  Good Bad  Good Bad  Bad  Bad  Good Bad  Good Good Good Bad
## [421] Good Good Bad  Good Good Good Good Bad  Good Good Bad  Good Good Good Good
## [436] Good Good Good Good Good Good Good Good Good Good Bad  Bad  Good Good Good
## [451] Good Good Bad  Bad  Good Good Bad  Good Good Bad  Bad  Good Good Good Bad
## [466] Good Good Good Good Good Bad  Bad  Good Good Good Bad  Good Good Good Good
## [481] Bad  Good Bad  Good Good Bad  Good Good Bad  Bad  Good Good Good Bad  Good
## [496] Good Good Good Good Bad  Good Bad  Good Bad  Good Good Good Good Good Good
## [511] Bad  Bad  Good Bad  Good Good Bad  Good Good Good Good Good Bad  Good Good
## [526] Bad  Good Good Bad  Good Good Good Good Good Good Good Good Bad  Good Good
## [541] Bad  Bad  Good Good Good Good Good Good Bad  Good Good Good Good Good Good
## [556] Good Bad  Good Good Bad  Bad  Bad  Good Good Good Bad  Bad  Good Good Good
## [571] Bad  Good Good Good Good Bad  Good Bad  Good Good Good Good Good Bad  Bad
## [586] Good Good Good Bad  Good Good Bad  Bad  Good Good Good Good Good Good Good
## Levels: Bad Good
```

11. Examine the predicted out object with **str( )**. Then, calculate a **confusion matrix** using the **table()** function.

```r
svm_pred <- predict(svm_model, testSet)

table(svm_pred, testSet$Class)
```

```
##
## svm_pred Bad Good
##     Bad   31   27
##     Good 149  393
```

12. Interpret the confusion matrix and in particular calculate the overall **accuracy** of the model. The

diag( ) command can be applied to the results of the table command you ran in the previous step. You can also use **sum( )** to get the total of all four cells.

```
(38+375)/(38+45+142+375)
```

```
## [1] 0.6883333
```

```
sum(diag(table(svm_pred, testSet$Class)))/sum(table(svm_pred, testSet$Class))
```

```
## [1] 0.7066667
```

13. Check you calculation with the **confusionMatrix()** function in the **caret** package.

```
confusionMatrix(svm_pred, testSet$Class)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Bad Good
##       Bad   31   27
##       Good 149  393
##
##                Accuracy : 0.7067
##                  95% CI : (0.6684, 0.7428)
##     No Information Rate : 0.7
##     P-Value [Acc > NIR] : 0.3797
##
##                   Kappa : 0.1339
##
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 0.17222
##             Specificity : 0.93571
##          Pos Pred Value : 0.53448
##          Neg Pred Value : 0.72509
##              Prevalence : 0.30000
##          Detection Rate : 0.05167
##    Detection Prevalence : 0.09667
##       Balanced Accuracy : 0.55397
##
##        'Positive' Class : Bad
##
```